The **with** clause introduced in SQL:1999, is currently supported only by some databases.

We could have written the above query by using a nested subquery in either the **from** clause or the **where** clause. However, using nested subqueries would have made the query harder to read and understand. The **with** clause makes the query logic clearer; it also permits a view definition to be used in multiple places within a query.

For example, suppose we want to find all branches where the total account deposit is less than the average of the total account deposits at all branches. We can write the query using the **with** clause as follows.

**with** *branch-total* (*branch-name*, *value*) **as**
    **select** *branch-name*, **sum**(*balance*)
    **from** *account*
    **group by** *branch-name*
**with** *branch-total-avg*(*value*) **as**
    **select avg**(*value*)
    **from** *branch-total*
**select** *branch-name*
**from** *branch-total*, *branch-total-avg*
**where** *branch-total.value* $>=$ *branch-total-avg.value*

We can, of course, create an equivalent query without the **with** clause, but it would be more complicated and harder to understand. You can write the equivalent query as an exercise.

# 4.9 Modification of the Database

We have restricted our attention until now to the extraction of information from the database. Now, we show how to add, remove, or change information with SQL.

## 4.9.1 Deletion

A delete request is expressed in much the same way as a query. We can delete only whole tuples; we cannot delete values on only particular attributes. SQL expresses a deletion by

**delete from** $r$
**where** $P$

where $P$ represents a predicate and $r$ represents a relation. The **delete** statement first finds all tuples $t$ in $r$ for which $P(t)$ is true, and then deletes them from $r$. The **where** clause can be omitted, in which case all tuples in $r$ are deleted.

Note that a **delete** command operates on only one relation. If we want to delete tuples from several relations, we must use one **delete** command for each relation.

The predicate in the **where** clause may be as complex as a **select** command's **where** clause. At the other extreme, the **where** clause may be empty. The request

**delete from** *loan*

deletes all tuples from the *loan* relation. (Well-designed systems will seek confirmation from the user before executing such a devastating request.)

Here are examples of SQL delete requests:

- Delete all account tuples in the Perryridge branch.

  **delete from** *account*
  **where** *branch-name* = 'Perryridge'

- Delete all loans with loan amounts between $1300 and $1500.

  **delete from** *loan*
  **where** *amount* **between** 1300 **and** 1500

- Delete all account tuples at every branch located in Needham.

  **delete from** *account*
  **where** *branch-name* **in** (**select** *branch-name*
  **from** *branch*
  **where** *branch-city* = 'Needham')

  This **delete** request first finds all branches in Needham, and then deletes all *account* tuples pertaining to those branches.

Note that, although we may delete tuples from only one relation at a time, we may reference any number of relations in a **select-from-where** nested in the **where** clause of a **delete**. The **delete** request can contain a nested **select** that references the relation from which tuples are to be deleted. For example, suppose that we want to delete the records of all accounts with balances below the average at the bank. We could write

**delete from** *account*
**where** *balance* < (**select avg** (*balance*)
**from** *account*)

The **delete** statement first tests each tuple in the relation *account* to check whether the account has a balance less than the average at the bank. Then, all tuples that fail the test—that is, represent an account with a lower-than-average balance—are deleted. Performing all the tests before performing any deletion is important—if some tuples are deleted before other tuples have been tested, the average balance may change, and the final result of the **delete** would depend on the order in which the tuples were processed!

### 4.9.2 Insertion

To insert data into a relation, we either specify a tuple to be inserted or write a query whose result is a set of tuples to be inserted. Obviously, the attribute values for inserted tuples must be members of the attribute's domain. Similarly, tuples inserted must be of the correct arity.

The simplest **insert** statement is a request to insert one tuple. Suppose that we wish to insert the fact that there is an account A-9732 at the Perryridge branch and that is has a balance of $1200. We write

```
insert into account
    values ('A-9732', 'Perryridge', 1200)
```

In this example, the values are specified in the order in which the corresponding attributes are listed in the relation schema. For the benefit of users who may not remember the order of the attributes, SQL allows the attributes to be specified as part of the **insert** statement. For example, the following SQL **insert** statements are identical in function to the preceding one:

```
insert into account (account-number, branch-name, balance)
    values ('A-9732', 'Perryridge', 1200)

insert into account (branch-name, account-number, balance)
    values ('Perryridge', 'A-9732', 1200)
```

More generally, we might want to insert tuples on the basis of the result of a query. Suppose that we want to present a new $200 savings acocunt as a gift to all loan customers of the Perryridge branch, for each loan they have. Let the loan number serve as the account number for the savings account. We write

```
insert into account
    select loan-number, branch-name, 200
    from loan
    where branch-name = 'Perryridge'
```

Instead of specifying a tuple as we did earlier in this section, we use a **select** to specify a set of tuples. SQL evaluates the **select** statement first, giving a set of tuples that is then inserted into the *account* relation. Each tuple has a *loan-number* (which serves as the account number for the new account), a *branch-name* (Perryridge), and an initial balance of the new account ($200).

We also need to add tuples to the *depositor* relation; we do so by writing

```
insert into depositor
    select customer-name, loan-number
    from borrower, loan
    where borrower.loan-number = loan.loan-number and
        branch-name = 'Perryridge'
```

This query inserts a tuple (*customer-name*, *loan-number*) into the *depositor* relation for each *customer-name* who has a loan in the Perryridge branch with loan number *loan-number*.

It is important that we evaluate the **select** statement fully before we carry out any insertions. If we carry out some insertions even as the **select** statement is being evaluated, a request such as

**insert into** *account*
  **select** *
  **from** *account*

might insert an infinite number of tuples! The request would insert the first tuple in *account* again, creating a second copy of the tuple. Since this second copy is part of *account* now, the **select** statement may find it, and a third copy would be inserted into *account*. The **select** statement may then find this third copy and insert a fourth copy, and so on, forever. Evaluating the **select** statement completely before performing insertions avoids such problems.

Our discussion of the **insert** statement considered only examples in which a value is given for every attribute in inserted tuples. It is possible, as we saw in Chapter 3, for inserted tuples to be given values on only some attributes of the schema. The remaining attributes are assigned a null value denoted by *null*. Consider the request

**insert into** *account*
  **values** ('A-401', *null*, 1200)

We know that account A-401 has $1200, but the branch name is not known. Consider the query

**select** *account-number*
**from** *account*
**where** *branch-name* = 'Perryridge'

Since the branch at which account A-401 is maintained is not known, we cannot determine whether it is equal to "Perryridge".

We can prohibit the insertion of null values on specified attributes by using the SQL DDL, which we discuss in Section 4.11.

### 4.9.3 Updates

In certain situations, we may wish to change a value in a tuple without changing *all* values in the tuple. For this purpose, the **update** statement can be used. As we could for **insert** and **delete**, we can choose the tuples to be updated by using a query.

Suppose that annual interest payments are being made, and all balances are to be increased by 5 percent. We write

**update** *account*
**set** *balance* = *balance* * 1.05

The preceding update statement is applied once to each of the tuples in *account* relation.

If interest is to be paid only to accounts with a balance of $1000 or more, we can write

```
update account
set balance = balance * 1.05
where balance >= 1000
```

In general, the **where** clause of the **update** statement may contain any construct legal in the **where** clause of the **select** statement (including nested **select**s). As with **insert** and **delete**, a nested **select** within an **update** statement may reference the relation that is being updated. As before, SQL first tests all tuples in the relation to see whether they should be updated, and carries out the updates afterward. For example, we can write the request "Pay 5 percent interest on accounts whose balance is greater than average" as follows:

```
update account
set balance = balance * 1.05
where balance > select avg (balance)
                from account
```

Let us now suppose that all accounts with balances over $10,000 receive 6 percent interest, whereas all others receive 5 percent. We could write two **update** statements:

```
update account
set balance = balance * 1.06
where balance > 10000

update account
set balance = balance * 1.05
where balance <= 10000
```

Note that, as we saw in Chapter 3, the order of the two **update** statements is important. If we changed the order of the two statements, an account with a balance just under $10,000 would receive 11.3 percent interest.

SQL provides a **case** construct, which we can use to perform both the updates with a single **update** statement, avoiding the problem with order of updates.

```
update account
set balance = case
                when balance <= 10000 then balance * 1.05
                else balance * 1.06
              end
```

The general form of the case statement is as follows.

**case**
  **when** $pred_1$ **then** $result_1$
  **when** $pred_2$ **then** $result_2$
  . . .
  **when** $pred_n$ **then** $result_n$
  **else** $result_0$
**end**

The operation returns $result_i$, where $i$ is the first of $pred_1, pred_2, \ldots, pred_n$ that is satisfied; if none of the predicates is satisfied, the operation returns $result_0$. Case statements can be used in any place where a value is expected.

### 4.9.4 Update of a View

The view-update anomaly that we discussed in Chapter 3 exists also in SQL. As an illustration, consider the following view definition:

**create view** *loan-branch* **as**
  **select** *branch-name, loan-number*
  **from** *loan*

Since SQL allows a view name to appear wherever a relation name is allowed, we can write

**insert into** *loan-branch*
  **values** ('Perryridge', 'L-307')

SQL represents this insertion by an insertion into the relation *loan*, since *loan* is the actual relation from which the view *loan-branch* is constructed. We must, therefore, have some value for *amount*. This value is a null value. Thus, the preceding **insert** results in the insertion of the tuple

('L-307', 'Perryridge', *null*)

into the *loan* relation.

As we saw in Chapter 3, the view-update anomaly becomes more difficult to handle when a view is defined in terms of several relations. As a result, many SQL-based database systems impose the following constraint on modifications allowed through views:

- A modification is permitted through a view only if the view in question is defined in terms of one relation of the actual relational database—that is, of the logical-level database.

Under this constraint, the **update**, **insert**, and **delete** operations would be forbidden on the example view *all-customer* that we defined previously.